olsrd 패킷 구현 백서

공식 v0.6.8 소스의 송신 경로와 메시지 바이트 형식

발행일 2026 년 10 월 20 일 | 판 1.0 | 대상 olsrd 0.6.8

분석 기준

이 백서는 OLSR 표준 문서를 요약하는 대신 OLSR/olsrd 저장소의 공식 v0.6.8 태그를 분석한다. 기준 커밋은 51ac86b22ee2bdb1077c889859ad3ca5a39aa079 다. v0.6.8.1, v0.6.8-eol, 최신 master, olsrd2 는 다른 분석 대상이다. [O01]

기본 설정에서 LinkQualityLevel 은 2, LinkQualityAlgorithm 은 etx_ff, LinkQualityFishEye 는 1 이다. 따라서 보통의 송신 집합은 LQ_HELLO 201, LQ_TC 202, MID 3, HNA 4 이고 HELLO 1 과 TC 2 는 링크 품질 기능을 끈 경로에서 사용된다. 실제 발생 여부는 인터페이스와 광고할 정보에 따라 달라진다. [O02, O03]

주제	핵심 결과
공통 형식	UDP 698 기본값, 4byte 패킷 헤더와 복수 메시지
링크 품질 확장	201 과 202 에 주소별 LQ 와 NLQ 및 예약 2byte
TC 구현 차이	예약 2byte 대신 lower 와 upper border 를 사용
Smart Gateway	새 message type 없이 HNA netmask 공간에 정보 삽입
플러그인	10~13, 130, 132, 171 과 별도 UDP 및 관리 TCP
소스 특이점	mdns 와 p2pd IPv6 길이식 및 secure 의 ABI 의존성

본문의 코드 관련 문장은 커밋에 고정한 참고 문헌 O01~O27 로 추적할 수 있다. 패킷 예제는 설명용 합성 데이터이며 실제 장비에서 캡처한 트래픽이 아니다.

목차

1 범위와 소스 증거를 읽는 방법 3
2 전체 송신 유형 목록 4
3 UDP 외피와 패킷 공통 헤더 5
4 메시지 공통 헤더와 유효시간 6
5 기본 HELLO 메시지 유형 1 7
6 LQ HELLO 메시지 유형 201 8
7 링크 품질 LQ 와 NLQ 및 ETX 9
8 기본 TC 메시지 유형 2 10
9 LQ TC 메시지 유형 202 와 단편 범위 11
10 MID 와 HNA 메시지 유형 3 과 4 12
11 Smart Gateway HNA 확장 13
12 생성과 묶음 및 재전파 경로 14
13 기본 타이머와 설정별 송신 차이 15
14 secure 플러그인 메시지 유형 10 부터 13 16
15 nameservice 메시지 유형 130 17
16 mdns 와 p2pd 메시지 유형 132 18
17 PUD 위치 메시지 유형 171 19
18 BMF 와 관리 및 기타 플러그인 트래픽 20
19 합성 패킷 바이트 예제 21
20 캡처와 구현 판별 절차 22
참고 문헌과 코드 위치 23

1 범위와 소스 증거를 읽는 방법

대상 범위는 코어가 생성하는 여섯 메시지 유형, 같은 트리를 통해 중계되는 메시지, 저장소에 포함된 플러그인의 독자 메시지 및 부가 네트워크 트래픽이다. 운영체제의 ARP·ND 와 라우팅 결과로 전달되는 일반 IP 데이터는 OLSR 제어 메시지와 구분한다. 플러그인은 로드와 설정이 있어야 동작하므로 저장소의 모든 기능이 기본 실행에서 동시에 송신된다고 보지 않는다.

소스 영역	읽을 이유
src/olsr_protocol.h	기본 message type 과 packed 구조체
src/build_msg.c 및 generate_msg.c	기본 메시지 구성과 송신 예약
src/lq_packet.c 및 lq_plugin_default_ff.c	확장 메시지 직렬화와 실제 LQ 4byte
src/net_olsr.c 및 parser.c	패킷 묶음, 송신 후크, 길이 검사와 분배
src/olsr.c 및 process_package.c	중계 조건과 설정별 파서 등록
src/gateway.c 및 gateway.h	Smart Gateway HNA 부호화
lib 아래 플러그인	메시지 번호, 캡슐화와 관리 포트

C 구조체의 메모리 크기가 언제나 전송 길이라는 가정은 피한다. 기본 구조체에는 가변 배열을 표현하는 [1] 원소가 있고, 일부 코드는 별도 직렬화를 한다. secure 는 time_t 와 sizeof 를 사용하므로 ABI 영향을 받는다. 코드에서 직접 확인한 사실과 실행 환경에서 확인해야 하는 조건을 구분한다.

재현 가능한 버전 식별

```
Repository  https://github.com/OLSR/olsrd
Tag         v0.6.8
Commit      51ac86b22ee2bdb1077c889859ad3ca5a39aa079
Archive     https://codeload.github.com/OLSR/olsrd/zip/refs/tags/v0.6.8
```

수집한 ZIP 의 SHA256 은 sources/provenance.json 에 기록했다. ZIP 의 해시는 원격 서버의 압축 재생성에 따라 달라질 수 있으므로, 소스 정체성의 주 기준은 고정 커밋과 개별 파일 해시다.

2 전체 송신 유형 목록

값 또는 전송	이름	발생 조건과 범위
1	HELLO	LinkQualityLevel 0 의 이웃 상태 광고
2	TC	LinkQualityLevel 0 의 토폴로지 광고
3	MID	추가 인터페이스 주소가 있을 때
4	HNA	외부 네트워크 또는 게이트웨이 광고
201	LQ_HELLO	LQ 활성 상태의 이웃 품질 광고
202	LQ_TC	LQ 활성 상태의 토폴로지 품질 광고
10 11 12 13	secure	서명 메시지와 challenge 교환
130	nameservice	이름, DNS forwarder, 서비스와 위치 등의 광고
132	mdns 또는 p2pd	선택한 IP 데이터의 OLSR 캡슐화
171	PUD	GPS 위치 갱신
UDP 50698	BMF	별도 BMF 캡슐화 데이터
UDP 5354	mdns router election	로컬 라우터 선출 메시지
UDP 2240 2241 2242	PUD 외부 인터페이스	NMEA 와 uplink 또는 downlink 기본값
TCP 관리 포트	txtinfo 등	요청한 클라이언트에 상태 제공

이 표는 저장소 검색에서 확인한 고유 메시지 생성 경로를 묶었다. 같은 132 를 mdns 와 p2pd 가 각각 사용하므로 번호만으로 어느 플러그인인지 구별할 수 없다. 목적지, 내부 IP 데이터와 설정을 확인해야 한다. [O13~O22]

src/olsr_protocol.h 의 MAX_MESSAGE 4 는 기본 메시지 정의 옆의 상수다. 그 값이 링크 품질 확장이나 플러그인 메시지가 존재하지 않는다는 뜻은 아니다. 실제 수신 분배는 등록한 파서 함수를 순회한다. [O02, O06]

3 UDP 외피와 패킷 공통 헤더

기본 olsrport 는 698 이며 IPv4 는 인터페이스의 브로드캐스트 설정을, IPv6 는 인터페이스의 멀티캐스트 설정을 사용한다. 목적지 주소는 실행 설정과 인터페이스를 확인한다. IP protocol number 698 로 식별하는 프로토콜이 아니다. UDP 포트 값과 IP 프로토콜 번호는 서로 다른 필드다. [O03, O05]

UDP payload 오프셋	길이	필드
0~1	2 bytes	Packet Length, 이 OLSR UDP payload 전체 길이
2~3	2 bytes	Packet Sequence Number, 송신 인터페이스별 증가
4 이후	가변	하나 이상의 OLSR 메시지

net_output 은 현재 버퍼에 패킷 sequence 와 길이를 넣고 플러그인의 packet transform 을 호출한 뒤 sendto 경로로 보낸다. secure 같은 후크는 길이와 내용을 추가로 바꿀 수 있다. 실제 패킷 길이는 후크 이후의 결과로 검사해야 한다. [O05]

패킷과 메시지 단위

한 UDP 데이터그램에는 서로 다른 message type 이 여러 개 들어갈 수 있다. 메시지의 sequence 는 메시지 생성자의 순번이고 패킷 sequence 는 바로 앞 송신자의 인터페이스 순번이다. 중계 노드는 메시지를 새 UDP 패킷에 다시 묶으므로 두 sequence 를 같다고 기대하지 않는다.

```
IP | UDP | Packet length 2 | Packet sequence 2
         | Message header | Message body
         | Message header | Message body ...
```

다중바이트 정수는 일반 공통 헤더에서 network byte order 를 사용한다. IPv4 와 IPv6 는 각각 4 바이트와 16 바이트 주소를 사용하되, 외부 IP 버전만으로 모든 플러그인의 내부 주소 크기까지 추정하면 안 된다. nameservice 와 secure 의 형식은 별도로 설명한다.

4 메시지 공통 헤더와 유효시간

필드	IPv4 위치	IPv6 위치	크기
Type	0	0	1 byte
Vtime	1	1	1 byte
Message Size	2~3	2~3	2 bytes
Originator	4~7	4~19	4 또는 16 bytes
TTL	8	20	1 byte
Hop Count	9	21	1 byte
Message Sequence	10~11	22~23	2 bytes
Body 시작	12	24	가변

위 위치는 메시지 시작을 0 으로 삼는다. Message Size 는 공통 헤더를 포함하고 OLSR 패킷의 4 바이트 헤더는 제외한다. Originator 는 생성자의 main address 다. 중계 시 바로 전 IP 송신자는 바뀌어도 Originator 와 message sequence 는 메시지의 식별 정보를 유지한다. [O02, O04]

Vtime 과 Htime

Vtime 은 메시지를 보관할 유효기간, HELLO 의 Htime 은 광고 간격이다. 둘은 같은 8 비트 mantissa·exponent 부호화 함수를 사용한다. 상위 4 비트 a, 하위 4 비트 b 에 대해 시간은 초 단위 1/16 × (1 + a/16) × 2^b 다. 0x05 는 2 초, 0x48 은 20 초다. 값 0x00 도 이 수식에서는 0.0625 초이며 단순한 0 초 표현으로 처리하지 않는다. [O07]

src/mantissa.c 의 구현은 밀리초 정수 연산으로 변환한다. 설정한 실수 시간과 디코딩한 Vtime 이 항상 같지는 않다. 예를 들어 300 초를 이 함수로 부호화하면 0x2C, 복호화는 288 초다. 백서의 타이머 표는 설정 기본값이며 캡처의 부호화 결과는 따로 계산한다.

TTL 의 두 계층

메시지 TTL 은 OLSR 중계 범위를 제어한다. 외부 IP 헤더의 TTL 또는 hop limit 과 동일한 필드가 아니다. Hop Count 는 OLSR 메시지의 전달 횟수이며 라우터가 일반 데이터그램의 IP TTL 을 줄이는 동작과 구분한다.

5 기본 HELLO 메시지 유형 1

본문 위치	길이	내용
0~1	2 bytes	Reserved
2	1 byte	Htime
3	1 byte	Willingness
4 이후	블록마다 가변	Link Code 1, Reserved 1, Link Message Size 2, 이웃 주소들

HELLO 는 직접 이웃 링크의 상태와 이웃 종류를 알린다. Willingness 는 MPR 로 일할 의향을 전달하며 기본값은 3 이다. 일반 HELLO 는 TTL 1, Hop Count 0 으로 생성된다. LQ 기능을 끈 모드에서 type 1 의 생성과 파서 등록을 사용한다. [O02, O03, O08]

비트와 값	의미
Link Code bits 1~0	0 UNSPEC, 1 ASYM, 2 SYM, 3 LOST
Link Code bits 3~2	0 NOT_NEIGH, 1 SYM_NEIGH, 2 MPR_NEIGH
Link Code bits 7~4	기본 링크 코드에서 사용하는 정보가 아님
블록 size	블록 헤더 4byte 와 그 뒤 모든 이웃 주소 길이의 합

CREATE_LINK_CODE(status, link)는 link | (status << 2)다. 예를 들어 0x0A 는 SYM 링크이면서 MPR 이웃임을 나타낸다. 코드에 HIDE_LINK 4 같은 내부 상태가 있어도 링크 코드 추출은 하위 2 비트이므로 별도 표준 전송값으로 추가하지 않는다. [O02]

IPv4 이웃 주소 한 개는 4 바이트, IPv6 는 16 바이트다. 이웃 수를 별도 단일 count 필드에서 읽는 대신 블록 size 를 이용해 다음 블록으로 이동한다. 이웃이 없는 HELLO 는 본문 기본 4 바이트만 가질 수 있다.

6 LQ HELLO 메시지 유형 201

기본 etx_ff 는 type 201 을 사용한다. HELLO 의 Reserved, Htime, Willingness 와 Link Code 블록 형태는 유지하지만 각 이웃 주소 바로 뒤에 품질 데이터 4 바이트가 붙는다. type 만 바꾸고 기본 HELLO 의 주소 배열로 읽으면 두 번째 주소부터 정렬이 틀어진다. [O04, O09]

항목	IPv4 크기	IPv6 크기
메시지 공통 헤더	12	24
HELLO 본문 헤더	4	4
각 Link Code 블록 헤더	4	4
각 이웃 항목	주소 4 + 품질 4 = 8	주소 16 + 품질 4 = 20
OLSR 패킷 공통 헤더	전체 묶음에 4	전체 묶음에 4

```
Neighbor entry in etx_ff
Address [4 or 16 bytes]
LQ      [1 byte]
NLQ     [1 byte]
Zero    [2 bytes]
```

default_lq_serialize_hello_lq_pair_ff 는 LQ 와 NLQ 를 각각 1 바이트로 쓰고 마지막 2 바이트를 0 으로 쓴다. 내부 구조체의 모든 통계나 창 배열을 패킷에 복사하는 것이 아니다. 역직렬화는 두 품질 바이트를 읽고 16 비트를 건너뛴다. [O09]

이웃 블록이 한 개이고 그 안에 이웃 n 개가 있으면 IPv4 메시지 길이는 12+4+4+8n, IPv6 는 24+4+4+20n 이다. 패킷 전체는 여기에 4 바이트와 다른 메시지 길이를 더한다. 블록이 늘면 블록마다 4 바이트를 추가한다.

송신 전 묶음과 분할

serialize_lq_hello 는 현재 버퍼 잔여량과 예상 크기를 비교한다. 기존 데이터가 공간을 차지하면 먼저 flush 해 전체 HELLO 가 불필요하게 나뉘는 것을 줄인다. 그래도 공간이 부족하면 블록을 마감하고 메시지를 송신한 뒤 새 블록으로 계속한다. 고정한 MTU 이상의 모든 이웃 목록이 언제나 하나의 메시지로 나간다고 가정하지 않는다. [O04]

7 링크 품질 LQ 와 NLQ 및 ETX

로컬 노드 A 가 B 의 패킷을 얼마나 잘 수신했는지가 A 의 해당 링크 LQ 다. B 가 자기 HELLO 에 보고한 반대 방향 값을 받아 A 의 NLQ 를 갱신한다. default_lq_memorize_foreign_hello_ff 는 foreign 의 valueLq 를 local 의 valueNlq 에 저장한다. 캡처에서 LQ 와 NLQ 를 읽을 때 광고한 노드의 관점을 유지해야 한다. [O09]

값	정규화	의미
255	1.0	측정과 부호화 범위에서 최대 품질
204	0.8	부호화된 수신 품질 80%
0	0	유효 품질이 없거나 추정 초기 상태일 수 있음
LQ 와 NLQ	각 값 ÷ 255	방향별 측정값이며 임의 float 바이트열이 아님

etx_ff 의 링크 비용은 유효 범위에서 255×255 ÷ (LQbyte×NLQbyte)의 고정소수점 계산에 해당한다. LQ=255, NLQ=204 이면 ETX 는 1.25 다. 양방향 모두 204 라면 약 1.5625 다. 너무 낮은 품질은 LINK_COST_BROKEN 으로 처리하므로 단순히 분모 0 의 무한대만 설명하면 구현의 임계값을 놓친다.

어떤 수신으로 품질을 추정하는가

etx_ff 는 packet parser 후크로 들어온 OLSR 패킷의 인터페이스별 sequence 차이를 추적한다. 같은 sequence 는 중복으로 다루고, 256 을 넘는 큰 점프는 보정한다. 1 초 타이머에서 창 통계를 계산하며 HELLO 미수신 정보와 loss multiplier 도 반영한다. 일반 사용자 데이터의 실제 손실률을 직접 계수하는 기능으로 해석하지 않는다. [O09]

동일한 201 과 202 를 쓰더라도 다른 LQ 알고리즘의 내부 측정·평활 방식은 다를 수 있다. 이 백서의 바이트 설명과 비용 예제는 기본 etx_ff 를 기준으로 하며, 다른 알고리즘을 선택한 실제 구성은 그 handler 의 serialize 함수와 크기를 확인한다. [O10]

8 기본 TC 메시지 유형 2

본문 위치	길이	의미
0~1	2 bytes	ANSN Advertised Neighbor Sequence Number
2~3	2 bytes	기본 TC 의 Reserved
4 이후	주소 크기 × n	광고하는 이웃 main address 목록

기본 TC 는 네트워크 토폴로지를 알린다. 공통 message sequence 는 메시지 인스턴스의 순번이고 ANSN 은 광고한 이웃 집합의 변화를 구분하는 별도 값이다. 두 값이 항상 같이 증가하는 것은 아니다. LinkQualityLevel 0 경로에서 type 2 를 사용한다. [O08, O03]

TcRedundancy 는 광고 범위를 바꾼다. 0 이면 MPR selector, 1 이면 MPR 와 MPR selector, 2 이면 대칭 이웃을 광고하는 경로를 사용한다. 따라서 TC 를 언제나 자신이 고른 MPR 목록만 담는 패킷으로 설명하면 안 된다. 기본 상수 TC_REDUNDANCY 는 2 다. [O02, O04]

TC 송신과 미디어 전달의 관계

TC 는 데이터 경로를 계산할 제어 정보를 전파한다. 음성 RTP 나 다른 IP 데이터가 TC 안에 들어가 전달되는 구조는 아니다. olsrd 가 운영체제 라우팅 테이블을 갱신하면 일반 데이터 전달은 그 라우팅을 따른다. BMF·p2pd 같은 명시적 캡슐화 플러그인만 예외적 별도 경로를 제공한다.

빈 TC 나 마지막 광고의 생존은 로컬 이웃 변화와 타이머 처리에 따라 달라진다. 특정 시점에 type 2 가 보이지 않는다고 모든 토폴로지 광고가 사라졌다고 보지 않는다. 기본 LQ 모드에서는 type 202 를 찾아야 한다.

9 LQ TC 메시지 유형 202 와 단편 범위

본문 위치	크기	의미
0~1	2 bytes	ANSN
2	1 byte	lower_border
3	1 byte	upper_border
4 이후	항목마다 가변	이웃 주소 + LQ 1 + NLQ 1 + zero 2

type 202 는 type 2 의 Reserved 16 비트 자리에 lower_border 와 upper_border 를 사용한다. serialize_lq_tc 는 주소를 정렬하고 버퍼가 부족하면 단편별 경계값을 계산한다. 첫 조각의 왼쪽과 마지막 조각의 오른쪽은 0xFF 로 표시된다. 분할하지 않은 완전한 목록은 FF FF 가 된다. [O04]

중간 border 값은 이웃 두 주소가 달라지는 비트 위치를 바탕으로 calculate_border_flag 에서 계산한다. 이는 IPv4 fragmentation 의 offset 이나 단순한 단편 번호가 아니다. 같은 ANSN 의 부분 광고로부터 어떤 주소 범위를 갱신하는지 판단하는 구현 정보다. 표준 TC 예약값이 아니라고 무조건 손상으로 버리면 정상 olsrd 메시지를 놓친다.

선택과 필터

create_lq_tc 는 대칭 이웃만 후보로 삼고 TcRedundancy 에 따른 MPR 조건을 적용한다. 이웃으로의 best link 가 없거나 비용이 LINK_COST_BROKEN 이상이면 광고하지 않는다. 주소별 품질은 선택한 링크에서 복사한다. 캡처에 모든 이웃이 없다고 이웃 테이블 자체가 비어 있다고 단정하지 않는다. [O04, O09]

FishEye TTL

LinkQualityFishEye 가 켜져 있으면 create_lq_tc 의 반복 TTL 목록은 2, 8, 2, 16, 2, 8, 2, 255 다. 초기 인덱스가 음수이면 MAX_TTL 을 먼저 쓸 수 있다. 끄면 MAX_TTL 을 사용한다. 가까운 노드에 더 자주 토폴로지를 알리는 방식이므로, TTL 이 짧은 TC 가 보인다고 IP 계층 장애로 해석하지 않는다. [O04]

LQ TC 의 이웃 한 항목은 etx_ff 에서 IPv4 8 바이트, IPv6 20 바이트다. IPv4 에서 이웃 n 개의 메시지 길이는 16+8n 이며 OLSR 패킷 헤더는 별도로 4 바이트다.

10 MID 와 HNA 메시지 유형 3 과 4

유형	본문 형태	송신 목적
MID 3	추가 인터페이스 주소의 연속 목록	main address 와 다른 인터페이스 주소를 연결
HNA 4	Network Address 와 Netmask 쌍의 목록	OLSR 외부의 도달 가능한 네트워크를 광고

MID 의 본문에 별도 count 나 고정 4 바이트 하위 헤더는 없다. IPv4 주소 n 개면 본문은 4n, IPv6 는 16n 바이트다. 수신자는 메시지 길이에서 공통 헤더를 빼 주소 수를 구한다. 추가 인터페이스 주소가 없으면 MID 가 생기지 않는 구성이 자연스럽다. [O08]

HNA 쌍은 IPv4 에서 주소 4+netmask 4=8 바이트, IPv6 에서 16+16=32 바이트다. prefix length 한 바이트를 전송하는 형식이 아니다. 예를 들어 IPv4 192.0.2.0/24 는 C0 00 02 00 FF FF FF 00 이라는 주소·마스크 쌍으로 나타낼 수 있다. [O02, O08]

HNA 와 기본 경로

HNA 의 0.0.0.0/0 같은 인터넷 경로 광고는 수신자에게 외부 경로를 제공한다. 실제 NAT 나 터널 생성은 HNA 바이트 그 자체가 수행하지 않는다. Smart Gateway 를 켜면 netmask 의 비 prefix 부분에 추가 정보가 들어갈 수 있으므로 일반 마스크만 검사하는 도구와 결과가 다를 수 있다. [O11]

HNA 는 목록의 네트워크가 실제로 언제나 통신 가능함을 검증하는 메시지가 아니다. dyn_gw 와 dyn_gw_plain 같은 플러그인은 조건에 따라 로컬 HNA 를 추가·삭제할 수 있다. 패킷에서 새 message type 이 생기는 대신 type 4 의 내용이 바뀐다. [O23]

11 Smart Gateway HNA 확장

Smart Gateway 기본값은 꺼짐이다. 활성화하면 build_msg 의 HNA 경로가 olsr_modifiy_inetgw_netmask 를 호출하고 게이트웨이 정보를 netmask 에 반영한다. 독립적인 Smart Gateway message type 이나 별도 type 5 를 만들지 않는다. [O03, O08, O11]

확장 영역 상대 위치	이름	내용
0	PAD	0 으로 두어 확장 식별에 사용
1	FLAGS	bit0 속도, bit1 IPv4, bit2 IPv4 NAT, bit3 IPv6, bit4 IPv6 prefix
2	UPLINK	업링크 속도 부호값
3	DOWNLINK	다운링크 속도 부호값
4	V6PREFIXLEN	IPv6 외부 prefix 길이
5 이후	V6PREFIX	IPv6 외부 prefix 정보

표의 오프셋은 netmask 의 시작이라고 모든 경우에 고정하지 않는다. hna_mask_to_hna_pointer 는 prefix length 를 기준으로 비 prefix 부분의 위치를 정한다. IPv4 /0 에서는 시작부터 네 바이트를 사용할 수 있지만, IPv6 의 다른 게이트웨이 prefix 에서는 시작 위치와 사용 공간이 달라진다. [O11]

속도 부호화

일반 속도 바이트 v 는 (v >> 3)+1 에 10^(v & 7)을 곱한 kbit/s 값으로 복호화한다. 0 과 255 는 함수에서 별도 특수 처리하므로 일반식만 적용하지 않는다. 송신 인코더도 최대값과 십진수 크기에 맞춰 양자화한다. 실제 링크 대역폭 실측값 자체가 32 비트 정수로 전송되는 것은 아니다. [O11]

올바른 광고의 판단에는 네트워크 prefix 가 인터넷 게이트웨이 범위인지, PAD 가 0 인지, FLAGS 가 0 이 아닌지를 함께 사용한다. IPv6 prefix 필드는 해당 조건이 맞을 때만 유효하다. 마스크가 비연속이라는 사실 하나만으로 손상이라고 단정하지 않는다.

터널 데이터와 분리

Smart Gateway 선택과 운영체제 터널을 통한 일반 데이터는 HNA 제어 패킷과 별도 흐름이다. IP-in-IP 등의 데이터 패킷을 OLSR message type 으로 분류하면 안 된다. 플랫폼과 실제 터널 설정을 함께 확인한다.

12 생성과 묶음 및 재전파 경로

```
Timer or topology change
  -> build message or create_lq_hello/create_lq_tc
  -> serialize common header and body
  -> net_outbuffer_push
  -> net_output
  -> set packet sequence and length
  -> packet transform hooks
  -> UDP sendto
```

기본 메시지는 generate_msg.c 와 build_msg.c, 링크 품질 메시지는 lq_packet.c 에서 생성된다. MID 와 HNA 도 출력 버퍼에 합쳐질 수 있다. 송신 시점은 생성 타이머와 forwarding jitter, 버퍼 잔여량의 영향을 받으므로 HELLO 간격이 2 초라는 설정이 정확히 2.000 초의 UDP 주기를 뜻하지 않는다. [O04, O05, O08]

수신 처리와 중계의 구분

parser.c 는 packet 길이, 메시지 최소 길이, 4 바이트 정렬과 잔여량을 검사한 뒤 등록된 message parser 를 호출한다. parser 가 false 를 반환하면 해당 메시지의 forwarding 을 금지한다. 등록된 parser 가 없는 message type 도 조건에 따라 기본 forwarding 판단에 도달할 수 있다. 로컬에서 이해하지 못하는 메시지라고 언제나 폐기되는 구조는 아니다. [O06]

olsr_forward_message 는 바로 전 송신자를 MID 로 main address 에 대응시키고, 대칭 이웃인지, 자신을 MPR 로 선택한 이웃인지, 중복인지 등을 검사한다. 일반 재전파에서 message TTL 을 줄이고 Hop Count 를 늘린다. Originator 와 message sequence 를 새 로컬 값으로 바꾸는 동작과는 다르다. [O12]

Ether 모드의 예외

이 구현에는 TTL 이 2 미만이거나 TTL+Hop Count 가 255 를 넘는 경우를 따로 처리하는 경로가 있다. 해당 메시지는 non-ether 출력으로 전달하지 않으며 ether 출력에서는 같은 수신 인터페이스로 되돌리지 않는다. 일부 ether 전달에서는 TTL·Hop Count 변경을 건너뛴다. 표준식 TTL 설명만으로 이 버전의 모든 중계 동작을 설명하지 않는다. [O12]

13 기본 타이머와 설정별 송신 차이

설정 또는 상수	기본값	패킷에 미치는 영향
HelloInterval	2 초	HELLO 또는 LQ_HELLO 생성 간격
HelloValidityTime	20 초	HELLO Vtime 의 설정 기본값
TcInterval	5 초	TC 또는 LQ_TC 생성 간격
TcValidityTime	300 초	TC Vtime 설정값이며 부호화 시 양자화
MidInterval 와 HnaInterval	각 5 초	내용이 있는 MID 와 HNA 송신 예약
MidValidityTime 와 HnaValidityTime	각 300 초	광고 만료 시간 설정
HELLO TC MID HNA jitter	각 25% 상수	스케줄 분산에 사용
Pollrate	0.05 초	이벤트 처리 주기 기본값
Willingness	3	MPR 선정 의향
LinkQualityLevel	2	기본 201 과 202 경로
LinkQualityAlgorithm	etx_ff	품질 측정과 직렬화 handler
LinkQualityFishEye	1	TC 의 전파 TTL 순환 활성
TcRedundancy	2	대칭 이웃 광고 범위
SmartGateway	false	기본 HNA 확장 비활성

이 표는 소스의 설정 기본값과 인터페이스 기본값이다. 사용자가 제공한 conf 파일, 플랫폼 패치, 커맨드라인과 런타임 조건으로 달라질 수 있다. 유효시간이 음수인 자동 설정 경로에는 LQ aging 을 이용하는 계산도 있으므로 실제 구성값을 함께 보관한다. [O02, O03, O24]

이 버전의 TOP_HOLD_TIME, MID_HOLD_TIME, HNA_HOLD_TIME 은 간격의 60 배다. 오래된 OLSR 설명에서 자주 보이는 3 배 기본값을 그대로 대입하지 않는다. 기본 HNA 와 MID 의 300 초도 wire 에서는 mantissa·exponent 변환의 영향을 받는다. [O02, O07]

14 secure 플러그인 메시지 유형 10 부터 13

유형	소스 구조체	주요 내용
10	s_olsrmsg 및 sig_msg	서명 종류, 알고리즘, 예약, timestamp, signature
11	challengemsg	destination, challenge, signature
12	c_respmsg	destination, challenge, timestamp, res_sig 와 signature
13	r_respmsg	destination, timestamp, res_sig 와 signature

secure 는 packet transform 으로 add_signature 를 등록한다. 이 함수는 기존 패킷 끝에 type 10 메시지를 붙이고 packet length 를 갱신한다. 메시지 Vtime 0, TTL 1, Hop Count 0 을 설정한다. 별도 응답 교환은 이웃의 timestamp 와 challenge 상태를 확인하는 경로에 사용된다. [O13, O14]

암호 방식과 실제 전송 크기

USE_OPENSSL 경로는 SHA1 기반 20 바이트, 다른 경로는 MD5 기반 16 바이트 signature 를 정의한다. 이 구현은 입력 바이트 뒤에 공유 키를 붙인 checksum 계산이며, 표준 HMAC 이나 디지털 공개키 서명과 같다고 부르면 안 된다. 이름에 secure 가 있어도 페이로드를 암호화해 숨기는 기능으로 해석하지 않는다. [O13]

secure_messages.h 의 구조체는 time_t 와 C 구조체 정렬에 의존하고, add_signature 는 sizeof(struct s_olsrmsg)를 전송 길이에 사용한다. timestamp 에는 htonl(now.tv_sec)를 대입한다. 그러므로 모든 32·64 비트 ABI 에서 서명 메시지가 하나의 고정 길이라고 쓰지 않는다. 빌드의 sizeof 와 정렬, USE_OPENSSL 여부를 확인해야 한다. [O14]

특히 실제 add_signature 경로는 IPv4 형태의 s_olsrmsg 를 사용한다. 헤더에 IPv6 이름의 구조체가 있다는 사실만으로 동일한 안전성과 상호운용성을 보장할 수 없다. 본문은 이 코드를 현재 권장 암호 설계로 평가하거나 실제 취약성 악용을 시험한 결과가 아니다.

분석 시 확인

서명 후크 앞뒤의 길이를 비교하고 마지막 메시지의 size 와 ABI 를 대조한다. challenge 유형의 destination 은 공통 Originator 와 다른 역할이다. UDP 698 캡처만으로 이 메시지를 HNA 나 링크 품질 데이터로 오분류하지 않는다.

15 nameservice 메시지 유형 130

부분	형식	역할
namemsg	version 16bit, nr_names 16bit	버전과 항목 수
각 name 항목	type 16bit, len 16bit, union olsr_ip_addr	종류, 문자열 길이와 IP
항목 가변부	len bytes 의 문자열 뒤 4byte 정렬	호스트 이름이나 서비스 등
프로토콜 버전	NAME_PROTOCOL_VERSION 1	OLSR 메시지 type 과 다른 버전 필드

항목 type 은 0 HOST, 1 FORWARDER, 2 SERVICE, 3 LATLON, 4 MACADDR 다. 기본 emission interval 은 120 초, name valid time 은 1800 초다. 호스트 이름과 DNS 서버, 서비스·위치·MAC 정보를 여러 항목으로 한 메시지에 실을 수 있다. [O15]

주소 필드의 크기

create_packet 은 sizeof(struct name) 뒤에 문자열을 쓴다. 이 구조체의 IP 는 union olsr_ip_addr 이며 IPv6 주소를 포함해 16 바이트를 차지한다. 따라서 보통의 이 소스 ABI 에서 name 고정 부분은 20 바이트다. 코어 메시지의 IPv4 주소 크기 4 바이트를 그대로 대입해서는 안 된다. [O15, O25]

type 과 len 은 htons 를 통해 기록하고 IP union 은 복사한다. 문자열은 len 만큼 쓰며 그 다음 4 바이트 정렬 경계까지 0 을 채운다. len 에 정렬 패딩을 포함하지 않는다. 문자열 길이가 이미 4 의 배수이면 정렬 루프가 추가 바이트를 쓰지 않는 코드도 읽어야 한다. 문자열 종료 규칙을 임의로 더하지 않는다.

수신과 재전파

수신 parser 는 자기 자신이 생성한 메시지와 비대칭 이웃에서 온 메시지를 걸러낸다. 이름 정보를 갱신한 뒤 true 를 반환하면 코어의 forwarding 조건으로 이어진다. nameservice 는 보통 외부 DNS 질의 패킷 자체를 OLSR 에서 대신 전송하는 서비스가 아니라 이름과 forwarder 정보를 광고하는 플러그인이다. [O15]

16 mdns 와 p2pd 메시지 유형 132

mdns 의 MESSAGE_TYPE 과 p2pd 의 P2PD_MESSAGE_TYPE 은 모두 132 다. 두 플러그인은 선택한 IP 패킷을 OLSR 메시지 본문에 복사하고 4 바이트 경계로 패딩한다. 안쪽 데이터는 원래 IP 헤더를 포함하는 형태이므로 이를 독자적인 DNS 헤더가 공통 헤더 직후에 있다고 가정해서 읽지 않는다. [O16, O17]

항목	IPv4 생성 경로	IPv6 생성 경로의 소스 표현
type	132	132
body	내부 IP 패킷 복사와 0 패딩	내부 IP 패킷 복사와 0 패딩
size 계산	aligned_size + 12	aligned_size + 12 + 96
정상 코어 헤더 크기	12 바이트	24 바이트
TTL	플러그인 설정 또는 MAX_TTL	플러그인 설정 또는 MAX_TTL

IPv6 길이식의 불일치

v0.6.8 의 olsr_mdns_gen 과 olsr_p2pd_gen 은 IPv6 size 에 aligned_size + 12 + 96 을 쓴다. 이는 실제 공통 IPv6 헤더 크기 24 와 84 바이트 차이가 있다. 출력 버퍼에 넣는 길이도 같은 식으로 늘린다. 따라서 이 버전의 소스를 설명하면서 해당 코드를 올바른 24 바이트 헤더 계산으로 고쳐 적으면 실제 구현을 숨기게 된다. [O16, O17]

이 항목은 소스에서 확인한 길이 계산의 불일치다. 특정 장비에서의 실행, 메모리 내용, 패치 적용 여부나 악용 가능성을 실험한 결론은 아니다. 캡처에서 실제 message size 와 내부 IP total length, 정렬 패딩 뒤의 바이트를 비교해 해당 빌드를 판별해야 한다.

mdns 의 별도 라우터 선출

mdns 는 OLSR type 132 외에 UDP 5354 로 router election HELLO 도 보낼 수 있다. 목적지는 IPv4 224.0.0.2 또는 IPv6 ff02::2 다. RtElHelloPkt 는 4byte head, int ipFamily, union router_id, uint8 network_id 의 packed 구조다. ipFamily 의 C int 사용 때문에 이를 독립 표준 네트워크 정수로 가정하지 않는다. [O18]

17 PUD 위치 메시지 유형 171

PUD 는 위치 정보를 OLSR 메시지 type 171 로 전달한다. 이 태그의 PUD_WIRE_FORMAT_VERSION 은 3 이다. 플러그인 전체 버전, OLSR version 과 이 위치 데이터 wire format version 은 서로 다른 번호다. [O19]

본문 부분	크기	설명
version	1 byte	위치 wire format 버전 3
validityTime	1 byte	PUD 자체 유효시간 표현
present	4 bytes	GPS 항목과 ID·gateway 존재 비트
GpsInfo	120bit 즉 15bytes	시간, 위도, 경도, 고도, 속도, 방향과 HDOP
NodeInfo	선택적 가변	nodeIdType 1byte 와 유형별 nodeId

GPS 필드	비트 수	용도
time	17	자정 이후 초
lat	28	위도 부호값
lon	27	경도 부호값
alt	16	고도 부호값
speed	12	속도
track	9	이동 방향
hdop	11	정밀도 정보

고정 본문은 NodeInfo 를 제외한 21 바이트다. NodeInfo 는 present 의 ID 비트가 있을 때 붙으며, DNS, MAC, IPv4, IPv6, UUID 등 nodeIdType 에 따라 길이가 달라진다. 비트필드를 단순한 uint32 배열로 읽지 말고 wireFormat 의 getter 와 setter 및 endian 매크로를 따라야 한다. [O19]

외부 PUD 흐름

기본 UDP 포트는 NMEA 수신·송신 멀티캐스트 2240, uplink 2241, downlink 2242 다. Uplink 는 type 0 POSITION 과 type 1 CLUSTERLEADER 를 정의하고, 4 바이트 TLV 헤더에 type, length 와 IPv6 flag 를 둔다. CLUSTERLEADER 페이로드는 논리적으로 IPv4 10 바이트 또는 IPv6 34 바이트다. 전송 길이는 family 에 맞춘 함수 경로로 확인한다. [O19, O20]

18 BMF 와 관리 및 기타 플러그인 트래픽

BMF 는 OLSR 메시지 type 을 새로 붙이는 경로와 다르게 UDP 50698 을 사용한다. 캡슐화 헤더는 type 1byte, len 1byte, reserved 2bytes, crc32 4bytes 로 총 8 바이트다. BMF_ENCAP_TYPE 은 1, BMF_ENCAP_LEN 은 6 이며 TLV 의 len 값 6 을 전체 헤더 크기 6 바이트라고 해석하면 안 된다. [O21]

기능	기본 전송 또는 효과	분류
txtinfo	TCP 2006	요청에 대한 텍스트 상태 응답
jsoninfo	TCP 9090	JSON 상태 응답
httpinfo	TCP 1978	HTTP 상태 페이지
dot_draw 와 pgraph	TCP 2004	그래프 출력, 설정 시 포트 충돌 주의
legacy IPC frontend	TCP 1212, 기본 연결 수 0	별도 관리 인터페이스
quagga	ZAPI 연결	다른 라우팅 데몬과 경로 연동
dyn_gw 와 dyn_gw_plain	HNA 목록 변경	새 message type 생성이 아님
sgwdynspeed	게이트웨이 속도 상태 변경	Smart Gateway HNA 내용에 반영
arprefresh	ARP 캐시 관련 처리	독자 OLSR 메시지 생성 없음
watchdog 와 mini	로컬 상태 또는 예제 기능	독자 OLSR 메시지 생성 없음

관리 포트 값은 해당 플러그인의 초기값이며 구성으로 변경할 수 있다. 이 포트에서 나가는 응답은 메시 네트워크의 OLSR flooding 패킷이 아니다. quagga 는 연결 방식과 ZAPI 판에 따라 프레이밍이 달라질 수 있으므로 TCP 브로드캐스트 OLSR 메시지로 오분류하지 않는다. [O22, O23, O26]

ARP·ND, 커널 netlink, 로컬 파일·UNIX socket 과 IP 터널 트래픽은 패킷 관찰 위치에 따라 다르게 보인다. 모든 운영체제 보조 트래픽을 olsrd 가 직접 생성한 UDP 698 메시지로 합산하지 않는다.

19 합성 패킷 바이트 예제

```
IPv4 OLSR packet with one LQ_HELLO, total 32 bytes
00 20 10 00
c9 48 00 1c 0a 00 00 01 01 00 12 34
00 00 05 03
0a 00 00 0c
0a 00 00 02 ff cc 00 00


IPv4 OLSR packet with one LQ_TC, total 28 bytes
00 1c 10 01
ca 08 00 18 0a 00 00 01 ff 00 12 35
00 07 ff ff
0a 00 00 02 ff cc 00 00
```

첫 예제의 packet length 는 32, packet sequence 는 0x1000 이다. 메시지 type=201, size=28, originator=10.0.0.1, TTL=1, Hop Count=0, message sequence=0x1234 다. Vtime 0x48 은 20 초, Htime 0x05 는 2 초, willingness 는 3 이다. 블록 code 0x0A 는 SYM_LINK 와 MPR_NEIGH, block size 12 는 헤더 4 와 이웃 항목 8 을 뜻한다.

이웃은 10.0.0.2 이고 LQ=255, NLQ=204, 예약 2 바이트는 0 이다. 비용 예제는 1.25 다. 두 번째 예제는 type=202, size=24, Vtime=0x08 즉 16 초, ANSN=7, border FF FF, 같은 이웃 품질이다. 여기서 16 초는 설명용 선택이며 기본 TC 유효시간 300 초를 흉내 낸 값이 아니다.

기본 메시지 길이 계산

IPv4 메시지 예제	메시지 길이	단독 OLSR 패킷 길이
기본 HELLO 와 한 이웃 블록의 한 주소	12 + 4 + 4 + 4 = 24	28
LQ_HELLO 와 한 이웃 블록의 한 주소	12 + 4 + 4 + 8 = 28	32
기본 TC 와 한 이웃	12 + 4 + 4 = 20	24
LQ_TC 와 한 이웃	12 + 4 + 8 = 24	28
MID 와 추가 주소 한 개	12 + 4 = 16	20
HNA 와 네트워크 한 개	12 + 8 = 20	24

재현 스크립트는 위 예제와 기본 1·2·3·4, IPv6 201·202 를 생성하고 길이·정렬을 검증한다. 실행 중인 네트워크로 송신하지 않으며, 합성 PCAP 은 오프라인 분석을 위한 파일이다.

20 캡처와 구현 판별 절차

검사 단계	판별 내용
외부 IP 와 UDP	포트 설정, 브로드캐스트 또는 멀티캐스트, 캡처 위치
Packet Length 와 Sequence	UDP payload 실제 길이와 헤더 값, 송신 인터페이스별 흐름
각 Message Size	IPv4 12 또는 IPv6 24 이상, 4byte 정렬, 잔여 길이
type 201 또는 1	LinkQualityLevel 경로 판별
type 202 의 border	단편 범위와 완성 목록 판별
3 과 4	추가 주소, HNA 와 Smart Gateway 확장
10~13 130 132 171	로드한 플러그인과 실제 형식 대조
별도 UDP 와 TCP	BMF, PUD, router election 과 관리 응답 분리

단순 udp.port == 698 필터는 기본 제어 트래픽의 출발점이다. UDP 50698, 5354, 2240~2242 와 관리 포트는 필요할 때 별도로 수집한다. 포트 변경 설정도 확인해야 하므로 필터만으로 모든 플러그인 트래픽을 포괄한다고 보지 않는다.

버전 판별은 실행 파일의 버전 문자열, 설정 파일, 빌드 옵션과 실제 메시지 패턴을 함께 사용한다. 201 을 봤다는 사실만으로 정확히 0.6.8 이라고 결정할 수 없다. 같은 형식을 다른 릴리스와 포크도 사용할 수 있다.

정적 검증의 범위

이 백서의 검증은 고정 태그의 소스 읽기, 생성 함수와 파서 대조, 합성 예제 길이·필드 확인, 문서 렌더 검토다. 데몬을 컴파일해 실제 메시 네트워크에서 송수신한 실행 검증을 의미하지 않는다. mdns·p2pd 의 IPv6 길이식과 secure 의 ABI 조건은 특히 해당 빌드의 캡처가 추가 판단에 필요하다.

구현에 맞춘 분석에서는 source size 와 실제 wire size 를 끝까지 추적한다. 표준의 Reserved 를 구현이 다른 목적으로 쓸 수 있고, 플러그인은 새 메시지나 다른 포트를 사용할 수 있다. 이 문서의 고정 커밋 링크와 sources/provenance.json 을 함께 보관하면 이후 패치와의 차이를 재현할 수 있다.

참고 문헌과 코드 위치

[O01] 공식 v0.6.8 커밋

https://github.com/OLSR/olsrd/commit/51ac86b22ee2bdb1077c889859ad3ca5a39aa079

[O02] 기본 message type 과 구조체 및 타이머 src/olsr_protocol.h

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/src/olsr_protocol.h#L66

[O03] 설정 기본값 src/olsr_cfg.h

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/src/olsr_cfg.h#L59

[O04] LQ_HELLO LQ_TC 생성과 직렬화 및 border src/lq_packet.c

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/src/lq_packet.c#L69

[O05] 패킷 헤더와 송신 transform 후크 src/net_olsr.c

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/src/net_olsr.c#L312

[O06] 패킷과 메시지 길이 검사 및 파서 분배 src/parser.c

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/src/parser.c#L264

[O07] 유효시간 부호화와 복호화 src/mantissa.c

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/src/mantissa.c#L58

[O08] 기본 HELLO TC MID HNA 직렬화 src/build_msg.c

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/src/build_msg.c#L280

[O09] 기본 etx_ff 의 품질 계산과 직렬화 src/lq_plugin_default_ff.c

고정 커밋의 소스 원문 열기

[O10] LQ handler 와 알고리즘 등록 src/lq_plugin.c

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/src/lq_plugin.c#L1

[O11] Smart Gateway mask 와 속도 부호화 src/gateway.c

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/src/gateway.c#L123

[O12] MPR forwarding 과 Ether 예외 src/olsr.c

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/src/olsr.c#L314

[O13] secure 송신 후크와 checksum lib/secure/src/olsrd_secure.c

고정 커밋의 소스 원문 열기

[O14] secure 의 time_t 및 구조체 lib/secure/src/secure_messages.h

고정 커밋의 소스 원문 열기

[O15] nameservice 의 생성과 가변 항목 lib/nameservice/src/nameservice.c

고정 커밋의 소스 원문 열기

[O16] mdns 캡슐화와 IPv6 길이식 lib/mdns/src/mdns.c

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/lib/mdns/src/mdns.c#L230

[O17] p2pd 캡슐화와 IPv6 길이식 lib/p2pd/src/p2pd.c

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/lib/p2pd/src/p2pd.c#L455

[O18] mdns 라우터 선출 lib/mdns/src/RouterElection.c

고정 커밋의 소스 원문 열기

[O19] PUD wire format 3 의 필드 lib/pud/wireformat/include/OlsrdPudWireFormat/wireFormat.h

고정 커밋의 소스 원문 열기

[O20] PUD 외부 인터페이스 기본값 lib/pud/src/configuration.h

고정 커밋의 소스 원문 열기

[O21] BMF 캡슐화 헤더 lib/bmf/src/Packet.h

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/lib/bmf/src/Packet.h#L45

[O22] JSON 상태 포트 초기값 lib/jsoninfo/src/olsrd_plugin.c

고정 커밋의 소스 원문 열기

[O23] 함께 포함된 플러그인 소스 전체 lib

https://github.com/OLSR/olsrd/tree/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/lib

[O24] 인터페이스 메시지 기본 타이머 src/cfgparser/olsrd_conf.c

고정 커밋의 소스 원문 열기

[O25] 주소 union 정의 src/olsr_types.h

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/src/olsr_types.h#L112

[O26] legacy IPC 기본 포트 src/ipc_frontend.h

https://github.com/OLSR/olsrd/blob/51ac86b22ee2bdb1077c889859ad3ca5a39aa079/src/ipc_frontend.h#L61

[O27] LQ 모드별 파서 등록 src/process_package.c

고정 커밋의 소스 원문 열기

보조 비교 문헌은 RFC 3626 OLSR 이다. 이 백서의 wire format 결론은 위 v0.6.8 구현을 우선하며, RFC 와 다른 내용을 자동으로 표준 형식으로 교정하지 않는다.

[O28] RFC 3626 Optimized Link State Routing

https://www.rfc-editor.org/rfc/rfc3626.html